Licence MIASHS parcours MIAGE — Deuxième année

Rapport de projet Développement Web

RECOMÉDIA

Votre plateforme de recommandations culturelles

Enseignante référente : Mme. Faten Hedjaj

Auteur : 42013418 Rakotoarivelo Shelly-Linda

Projet réalisé du 31 janvier 2026 au 29 avril 2026

Code source complet :
`https://github.com/shellyvl/RecoMedia`

Remerciements

Je tiens à remercier ma professeure responsable de l'UE pour ses enseignements qui m'ont permis d'acquérir les bases solides du développement Web. Ce projet en totale autonomie a été l'occasion de mettre en pratique les concepts théoriques vus en cours et de me confronter aux réalités de la programmation "Full Stack" (de la conception de la base de données jusqu'à l'intégration du Front-end).

Table des matières

Remerciements **1**

1 Introduction **3**
1.1 Contexte du projet . . . 3
1.2 Genèse du projet : Le défi technique . . . 3
1.3 Le concept de RecoMédia . . . 3

2 Fonctionnalités **4**
2.1 Fonctionnalités de l'application . . . 4
2.2 Sécurité et Authentification . . . 4
2.3 Espace Public et Consultation . . . 4
2.4 Espace Privé et Gestion (CRUD) . . . 4
2.5 Espace Administration . . . 4

3 Conception **6**
3.1 Diagramme de cas d'utilisation . . . 6
3.1.1 Acteurs du système . . . 6
3.1.2 Description textuelle du cas : "Ajouter une recommandation" . . . 7
3.2 Diagramme de classes . . . 9
3.3 Diagrammes de séquence . . . 10
3.3.1 Scénario 1 : Authentification d'un membre . . . 10
3.3.2 Scénario 2 : Ajout d'une nouvelle recommandation . . . 11

4 Base de données et Requêtes SQL **12**
4.1 Dictionnaire des données détaillé . . . 12
4.2 Requêtes SQL Majeures (CRUD) . . . 13
4.3 Implémentations techniques remarquables . . . 13

5 Environnement technique et Structure du projet **15**
5.1 Outils et Technologies . . . 15
5.2 Architecture des fichiers (Arborescence) . . . 15

6 Interface du site et Expérience Utilisateur (UX) **16**
6.1 Catalogue public . . . 16
6.2 Espace Membre (Tableau de bord) . . . 17
6.3 Fiche détaillée et Avis . . . 18
6.4 Panel Administrateur . . . 18

7 Difficultés rencontrées et Solutions **19**
7.1 Défis d'infrastructure et serveur . . . 19
7.2 Défis de logique applicative et UX . . . 19

8 Conclusion et Perspectives **20**
8.1 Bilan du projet . . . 20
8.2 Perspectives d'évolution . . . 20

9 Bibliographie et Webographie **21**

1 Introduction

1.1 Contexte du projet

Ce projet a été réalisé dans le cadre de ma formation universitaire, avec pour objectif la conception et le développement complet d'une application web dynamique. Le cahier des charges m'imposait de construire un projet "Full Stack" de A à Z : modélisation d'une base de données relationnelle, développement Back-end, et création d'une interface Front-end interactive. Il s'agissait de mettre en pratique les concepts fondamentaux du web tout en garantissant la sécurité et la cohérence des données manipulées.

1.2 Genèse du projet : Le défi technique

Le choix de mon sujet s'est porté sur une plateforme de recommandations culturelles. Initialement, j'aurais pu me contenter d'un simple catalogue d'œuvres statique. Cependant, cette approche me semblait trop simple techniquement pour un projet universitaire de ce niveau.

Pour élever le défi, j'ai décidé d'orienter l'architecture autour des interactions sociales et de la gestion des droits. Le véritable enjeu technique n'était plus seulement d'afficher des données, mais de sécuriser un espace multi-utilisateurs. Cela impliquait d'implémenter un système d'authentification robuste (hachage des mots de passe), de gérer des sessions hiérarchisées (Visiteurs, Membres, Administrateurs), et de garantir l'intégrité de la base de données (modérations, suppressions en cascade) face aux actions imprévisibles des utilisateurs.

1.3 Le concept de RecoMédia

Pensé pour les amateurs de cinéma et de pop culture, le fruit de mon travail est "RecoMédia", votre bibliothèque culturelle personnelle. L'idée fondatrice est d'offrir un espace convivial d'échange qui s'articule autour de quatre objectifs majeurs : **Découvrir** de nouvelles œuvres, **Partager** ses propres recommandations, **Noter** les contenus via un système d'avis, et **Consulter** les profils des autres cinéphiles.

L'utilisateur ne se contente pas de consulter le catalogue : il en est l'acteur principal. Il peut enrichir la base de données via son tableau de bord personnalisé, laisser des avis détaillés avec un système de notation, et interagir avec les recommandations des autres membres. De son côté, l'administrateur dispose d'un panel de modération global pour maintenir la qualité des échanges. C'est cet équilibre entre l'autonomie laissée aux utilisateurs et le contrôle strict des données en Back-end qui définit l'identité de mon application.

2 Fonctionnalités

2.1 Fonctionnalités de l'application

Afin de répondre aux objectifs fixés, l'application intègre un ensemble de fonctionnalités réparties selon le niveau d'habilitation de l'utilisateur (Visiteur, Membre, Administrateur).

2.2 Sécurité et Authentification

- Création de compte avec vérification de la confirmation du mot de passe.
- Connexion sécurisée avec hachage cryptographique des mots de passe.
- Gestion des sessions PHP pour maintenir l'état de connexion à travers les différentes pages et protéger les routes privées.
- Déconnexion sécurisée détruisant les variables de session.

2.3 Espace Public et Consultation

- **Catalogue dynamique :** Parcourir l'ensemble des œuvres (Films, Séries, Animés) triées par date d'ajout.
- **Moteur de recherche :** Filtrer instantanément les contenus par titre ou par catégorie via la barre de recherche.
- **Fiches détaillées :** Consulter les informations complètes d'une œuvre (synopsis, genre, année, durée/saisons, affiche visuelle).
- **Lecture des avis :** Visualiser les notes et lire les critiques laissées par la communauté.
- **Profils publics :** Visiter les profils des autres membres pour consulter leurs statistiques et leurs dernières contributions.

2.4 Espace Privé et Gestion (CRUD)

- **Tableau de bord (CRUD) :** Ajouter de nouvelles recommandations via un formulaire dynamique (les champs s'adaptent selon s'il s'agit d'un film ou d'une série). Le membre peut également modifier ou supprimer ses propres contenus.
- **Gestion du profil personnel :** Éditer sa propre biographie et suivre l'évolution de ses statistiques (compteur d'œuvres ajoutées et d'avis postés).
- **Système d'évaluation :** Publier un avis textuel sur une œuvre, accompagné d'une note sur 5.
- **Modération personnelle :** Modifier la note et le texte de ses propres commentaires, ou les supprimer à tout moment.

2.5 Espace Administration

- **Panel de contrôle global :** Accéder à une interface de supervision réservée, listant tous les membres inscrits et toutes les œuvres de la base de données.
- **Modération des contenus :** Supprimer définitivement une œuvre de la plateforme.

— **Gestion des utilisateurs :** Bannir un membre du site. Cette action déclenche un script de "suppression en cascade" nettoyant automatiquement la base de tous ses avis et contenus associés.

3 Conception

La conception de RecoMédia s'appuie sur la modélisation UML (Unified Modeling Language) afin de structurer les interactions et l'architecture des données avant le développement.

3.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation est une étape cruciale de la conception. Il permet de formaliser les besoins du système en définissant les interactions entre les acteurs externes et les fonctionnalités de RecoMédia.

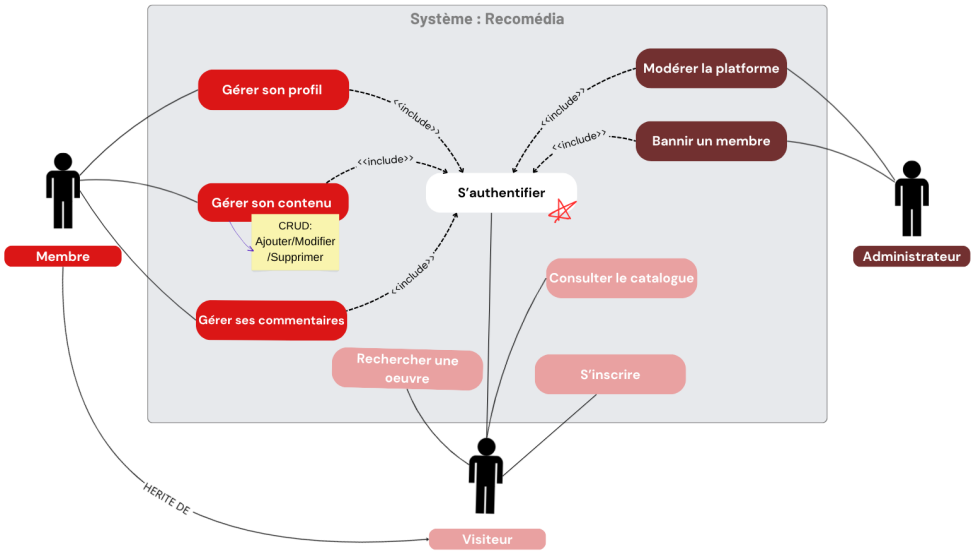

FIGURE 1 – Diagramme de cas d'utilisation de RecoMédia

3.1.1 Acteurs du système

Notre application fait intervenir trois acteurs compte tenu du déroulement et de la complémentarité des opérations :

Acteurs	Description
L'administrateur	1. Authentification 2. Modération globale : supprimer n'importe quel contenu ou commentaire 3. Gestion des utilisateurs : bannir un membre (suppression en cascade) 4. Déconnexion
Le Membre	Il bénéficie de la plupart des fonctions interactives : 1. Créer un compte (s'inscrire) 2. Authentification 3. Gestion de son profil (bio) 4. Gestion de ses recommandations : Ajouter, modifier, supprimer 5. Gestion de ses commentaires : Ajouter, modifier, supprimer 6. Déconnexion
Le Visiteur	1. Consulter le catalogue public et lire les avis 2. Effectuer des recherches d'œuvres

Table 1 – Description des Acteurs du système

3.1.2 Description textuelle du cas : "Ajouter une recommandation"

Conformément aux méthodologies de conception, voici le détail du scénario principal de l'application :

- **Acteur principal :** Membre connecté.
- **Pré-conditions :** L'utilisateur doit être authentifié et se trouver sur son tableau de bord.
- **Scénario nominal :**
  1. L'utilisateur accède au formulaire d'ajout.
  2. Il saisit les informations (Titre, Type, Synopsis, Image).
  3. Si le type choisi est "Série", des champs dynamiques (Saisons/Épisodes) apparaissent via JavaScript.
  4. L'utilisateur valide le formulaire.
  5. Le système vérifie les données et crée l'entrée dans la base MySQL.
- **Post-conditions :** L'œuvre est ajoutée au catalogue et associée à l'ID de l'utilisateur.

Pour compléter cette description, le diagramme d'activité suivant (Figure 2) illustre le flux logique du script, notamment les étapes de validation et de sécurisation des données avant l'insertion en base.

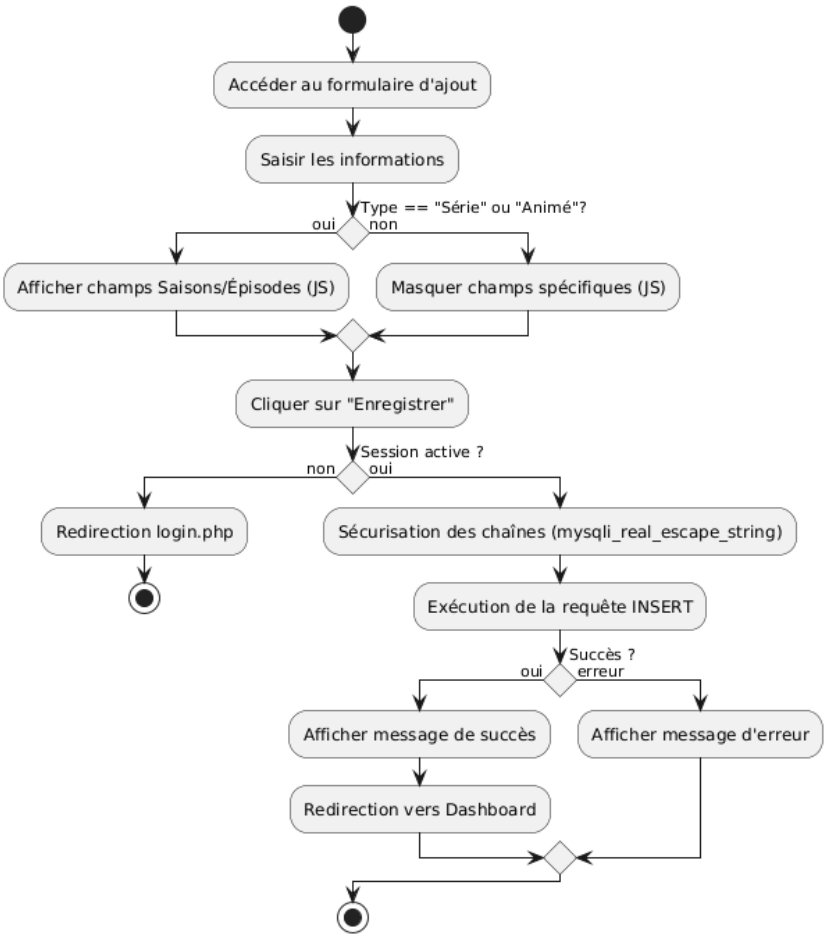

FIGURE 2 – Diagramme d'activité : Processus d'ajout de contenu

3.2 Diagramme de classes

Le diagramme de classes (Figure 3) représente la structure statique de la base de données de RecoMédia. Il permet de définir les entités, leurs attributs ainsi que les relations logiques qui les unissent. *(Le dictionnaire des données détaillé et l'implémentation physique sont présentés dans la Section 5).*

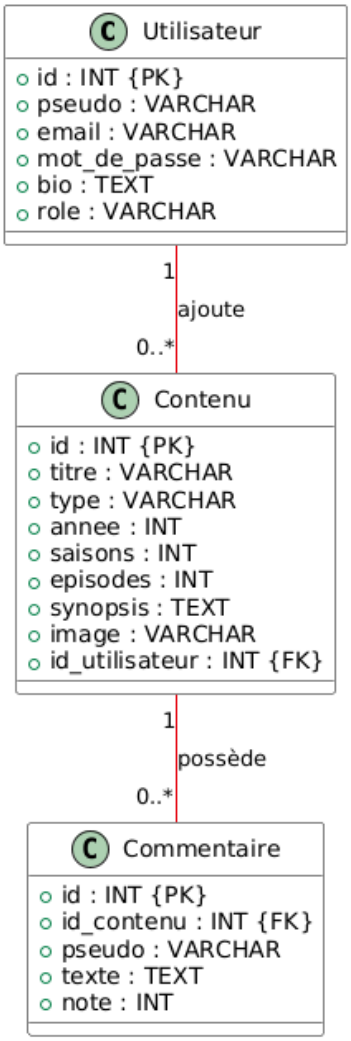

FIGURE 3 – Structure relationnelle de la base de données RecoMédia

Règles de gestion principales : L'intégrité du système est garantie par des relations de clés étrangères. Par exemple, un commentaire est obligatoirement rattaché à un contenu cible et à un utilisateur source. La suppression d'un compte entraîne automatiquement la *suppression en cascade* de ses contenus et avis associés.

3.3 Diagrammes de séquence

Les diagrammes de séquence illustrent la dynamique du système en montrant l'ordre chronologique des messages échangés entre l'utilisateur et les composants techniques. J'ai modélisé les deux processus fondamentaux du parcours utilisateur.

3.3.1 Scénario 1 : Authentification d'un membre

Ce premier diagramme illustre la logique de vérification lors de la connexion, incluant les alternatives (succès ou échec) en fonction de la validation du mot de passe haché.

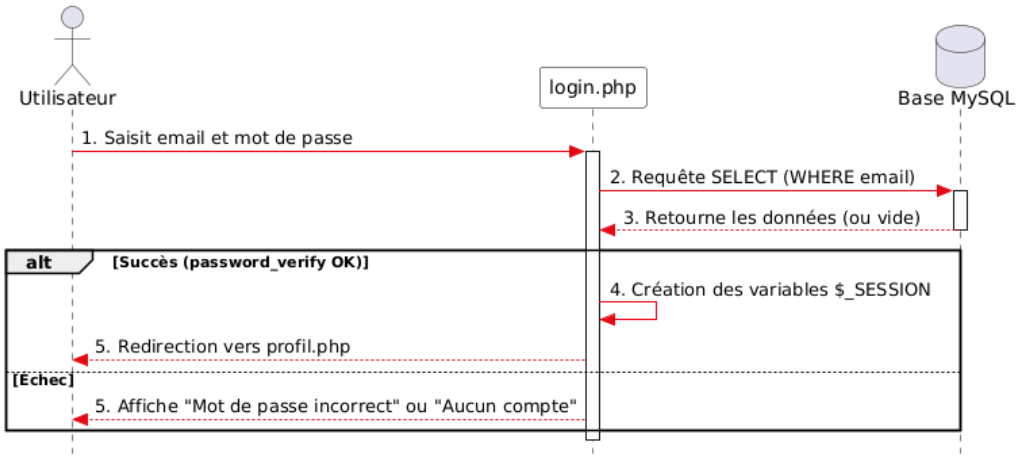

FIGURE 4 – Diagramme de séquence : Processus d'authentification

Pré condition :
L'utilisateur possède déjà un compte et se trouve sur la page de connexion.

Scénario normal :

1. L'utilisateur saisit son email et son mot de passe.
2. Le système interroge la base de données pour trouver l'email correspondant.
3. Si l'email existe, le système compare le mot de passe saisi avec l'empreinte hachée via la fonction PHP `password_verify()`.
4. Si la correspondance est validée, le système initialise les variables de session (`$_SESSION`).
5. L'utilisateur est identifié et redirigé vers son profil.

Post condition :
L'utilisateur est connecté, sa session est active, et il accède aux fonctionnalités privées.

3.3.2 Scénario 2 : Ajout d'une nouvelle recommandation

Une fois authentifié, l'utilisateur peut enrichir le catalogue. Ce diagramme détaille le flux d'insertion sécurisée d'une nouvelle œuvre.

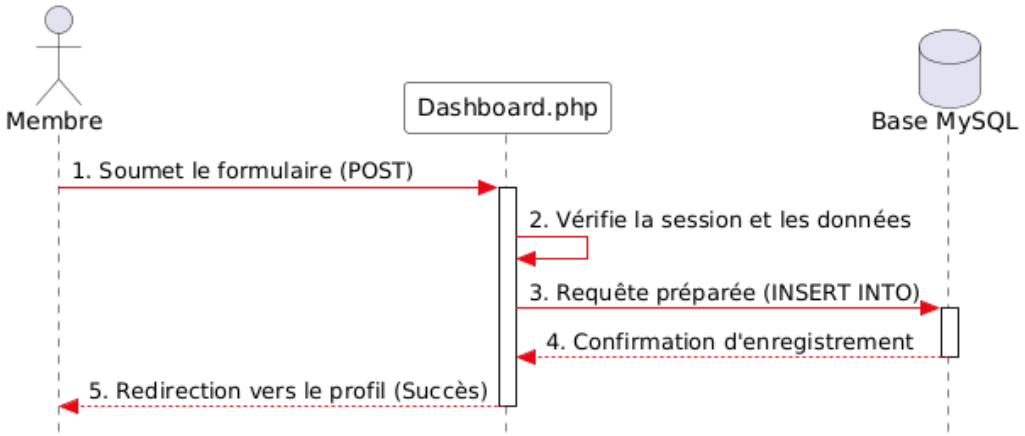

FIGURE 5 – Diagramme de séquence : Ajout d'une recommandation

Pré condition :
L'opération d'ajout de recommandation doit être demandée.
Le membre doit être préalablement authentifié (Session active).

Scénario normal :

1. Le membre remplit le formulaire d'ajout d'œuvre.
2. Le système (serveur PHP) vérifie la validité des champs saisis et confirme que la session est active.
3. Le membre valide la saisie (envoi des données via POST).
4. Le système exécute la requête SQL préparée (INSERT) vers la base de données MySQL.
5. Les données sont enregistrées avec succès dans la table `contenus`.
6. Le système affiche un message de confirmation et redirige le membre vers son tableau de bord.

Post condition :
L'œuvre est ajoutée au catalogue avec succès et rattachée à l'identifiant du membre.

4 Base de données et Requêtes SQL

La modélisation conceptuelle a été implémentée physiquement via le SGBD **MySQL**. L'interaction avec la base de données a été développée en PHP en utilisant l'extension **MySQLi**.

4.1 Dictionnaire des données détaillé

Pour garantir la pérennité de l'application, la base de données `recomedia` est divisée en trois tables principales, reliées par des clés étrangères.

Table 1 : utilisateurs

Attribut	Type	Rôle / Contrainte
id	INT	Clé primaire (auto-incrément)
pseudo	VARCHAR(50)	Nom d'affichage du membre (Unique)
email	VARCHAR(100)	Identifiant de connexion (Unique)
mot_de_passe	VARCHAR(255)	Empreinte cryptographique (Hachée)
role	VARCHAR(20)	Niveau d'accès (membre ou admin)
bio	TEXT	Biographie modifiable depuis le profil

Table 2 : contenus

Attribut	Type	Rôle / Contrainte
id	INT	Clé primaire (auto-incrément)
titre	VARCHAR(255)	Titre de l'œuvre
type	VARCHAR(50)	Catégorie (Film, Série, Animé)
genre	VARCHAR(100)	Genre de l'œuvre (Action, Drame...)
annee	INT	Année de sortie
duree	VARCHAR(50)	Durée du film ou des épisodes
saisons	INT	Nombre de saisons (spécifique aux séries)
episodes	INT	Nombre d'épisodes (spécifique aux séries)
image	VARCHAR(255)	URL de l'affiche de l'œuvre
synopsis	TEXT	Résumé de l'œuvre
note	INT	Note moyenne ou note de l'auteur (sur 5)
id_utilisateur	INT	Clé étrangère (auteur de la recommandation)

Table 3 : commentaires

Attribut	Type	Rôle / Contrainte
id	INT	Clé primaire (auto-incrément)
id_contenu	INT	Clé étrangère (Cible de l'avis)
pseudo	VARCHAR(50)	Clé étrangère (Auteur de l'avis)
texte	TEXT	Corps de l'avis
note	INT	Note attribuée (de 1 à 5)

4.2 Requêtes SQL Majeures (CRUD)

Voici un aperçu des requêtes codées dans l'application pour manipuler ces données, avec une mise en évidence de la syntaxe :

Lecture complexe (SELECT avec Jointure)

Pour l'affichage du panel administrateur, une jointure relationnelle est nécessaire afin de récupérer les contenus tout en affichant le pseudo de l'auteur :

```
SELECT contenus.*, utilisateurs.pseudo AS auteur
FROM contenus
LEFT JOIN utilisateurs ON contenus.id_utilisateur = utilisateurs.id
ORDER BY contenus.id DESC;
```

Insertion sécurisée (INSERT)

Lors de l'inscription d'un membre (`register.php`), le mot de passe est systématiquement haché via `password_hash()` avant l'insertion en base :

```
INSERT INTO utilisateurs (pseudo, email, mot_de_passe)
VALUES ('$pseudo', '$email', '$mdp_hache');
```

Suppression logique en cascade (DELETE)

Afin de préserver l'intégrité de la base lors du bannissement d'un membre (`supprimer_user.php`), un script PHP se charge de supprimer les données dans l'ordre hiérarchique pour éviter les données orphelines :

```
DELETE FROM commentaires WHERE pseudo = '$pseudo_a_bannir';
DELETE FROM contenus WHERE id_utilisateur = $id_a_bannir;
DELETE FROM utilisateurs WHERE id = $id_a_bannir;
```

4.3 Implémentations techniques remarquables

Au-delà des opérations de base (CRUD), plusieurs fonctionnalités avancées ont été développées pour sécuriser l'application et enrichir l'expérience utilisateur :

- **Sécurité cryptographique et Contrôle d'accès :** Les mots de passe ne sont jamais stockés en clair (sécurisés via `password_hash()` et `password_verify()`). De plus, chaque page sensible vérifie strictement la session active (`$_SESSION['user_id']`), avec un blocage supplémentaire empêchant l'escalade de privilèges vers le panel administrateur.
- **Prévention des failles web (SQLi et XSS) :** Toutes les requêtes sont protégées contre les injections SQL grâce au nettoyage des saisies par `mysqli_real_escape_string()`. Lors de l'affichage, les failles XSS sont neutralisées en convertissant les caractères spéciaux via `htmlspecialchars()`.
- **Moteur de recherche dynamique :** La page d'accueil intègre une barre de recherche fonctionnelle. Le script PHP capture la saisie en méthode GET et modifie la requête SQL initiale en y ajoutant une clause conditionnelle pour filtrer le catalogue :

```
$sql = "SELECT * FROM contenus ORDER BY id DESC";
if (isset($_GET['recherche']) && !empty($_GET['recherche'
    ])) {
    $recherche = mysqli_real_escape_string($conn, $_GET['
        recherche']);
    $sql = "SELECT * FROM contenus WHERE titre LIKE '%
        $recherche%' OR type LIKE '%$recherche%'";
}
```

— **Système d'avis dynamique :** Sur la page des œuvres (`detail.php`), l'affichage des notes s'adapte visuellement. Au lieu d'utiliser de multiples conditions lourdes, j'ai optimisé le code en utilisant la fonction native PHP `str_repeat()` pour générer dynamiquement la jauge d'étoiles :

```
echo str_repeat("★", $com['note']) . str_repeat("☆", 5 -
    $com['note']);
```

Des contrôles de session garantissent ensuite que seul l'auteur d'un avis peut le modifier ou le supprimer.

— **Requêtes statistiques avancées :** Les pages de profils utilisent des fonctions d'agrégation SQL (`COUNT(*)`) pour calculer et afficher en temps réel l'implication de la communauté (nombre d'avis postés et de recommandations ajoutées).

5 Environnement technique et Structure du projet

5.1 Outils et Technologies

Afin de développer cette plateforme, je me suis appuyée sur l'écosystème suivant :

- **Serveur et Logique (Back-end) :** PHP 8 couplé à un serveur local (XAMPP) pour le traitement des données et la gestion des sessions.
- **Base de données :** MySQL pour le stockage persistant, interrogé via `MySQLi`.
- **Interface (Front-end) :** HTML5, CSS3 (utilisation de variables globales et d'effets Glassmorphism) et Vanilla JavaScript pour rendre les formulaires interactifs.
- **Éditeur de code :** Visual Studio Code.

5.2 Architecture des fichiers (Arborescence)

Pour garantir la maintenabilité du code, le projet a été structuré de manière modulaire :

```
Recomedia/
|-- recomedia.sql         # Script d'exportation de la base de données
|-- index.php             # Page d'accueil (Catalogue public et recherche)
|-- connexion.php         # Script global de connexion à la BDD MySQLi
|
|-- /css
|   |-- style.css         # Feuille de style (Variables, Dark Mode, Glassmorphism)
|
|-- /js
|   |-- script.js         # Scripts dynamiques (Formulaire interactif, Bio)
|
|-- /images
|   |-- logo4.png         # Identité visuelle du site
|   |-- fond-cinema
|
\-- /pages                # Dossier regroupant la logique métier et les vues PHP
    |
    |   [Authentification]
    |-- login.php         # Traitement de connexion et création de session
    |-- register.php      # Inscription
    |-- logout.php        # Destruction de la session
    |
    |   [Espace Membres]
    |-- dashboard.php     # Tableau de bord (Ajout de contenus et gestion)
    |-- profil.php        # Espace personnel (Statistiques, modification Bio)
    |-- profil_public.php # Vue des statistiques d'un autre membre
    |-- modifier.php      # Formulaire de mise à jour d'un contenu
    |-- supprimer.php     # Script de suppression ciblée d'un contenu
    |-- detail.php        # Fiche d'une œuvre et système de commentaires
    |
    |-- [Espace Admin]
    |-- admin.php         # Panel de supervision (Listing global)
    |-- supprimer_user.php     # Script de bannissement avec suppression en cascade
```

6 Interface du site et Expérience Utilisateur (UX)

Pour garantir une expérience utilisateur optimale et immersive, j'ai conçu une interface graphique inspirée des salles de cinéma. L'intégration d'un thème sombre (Dark Mode) associé à des effets de transparence modernes (Glassmorphism) permet de mettre en valeur les affiches des films tout en reposant les yeux de l'utilisateur. L'architecture a été pensée pour séparer intuitivement les espaces publics de consultation des espaces privés de gestion (tableaux de bord).

6.1 Catalogue public

La page d'accueil (Figure 6) est accessible à tous. Elle génère dynamiquement les cartes des œuvres à partir de la base de données et intègre une barre de recherche.

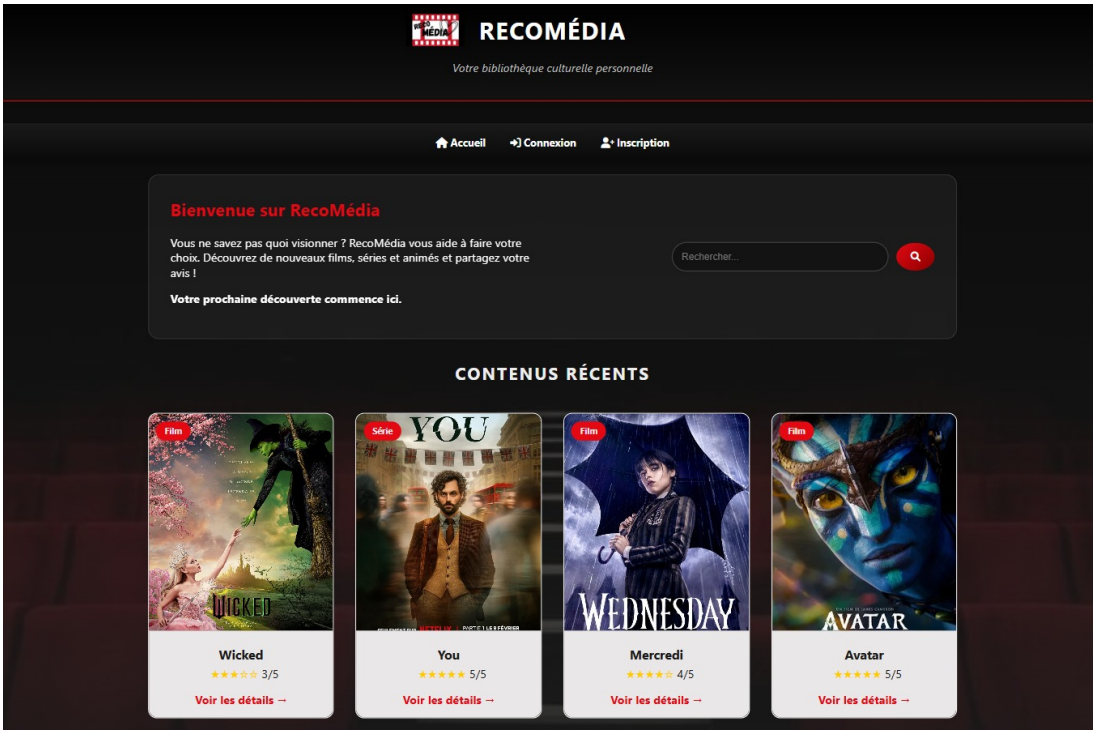

FIGURE 6 – Interface de la page d'accueil (Catalogue dynamique)

6.2 Espace Membre (Tableau de bord)

L'espace privé (Figure 7) permet aux utilisateurs d'interagir avec la base de données via un formulaire d'ajout rendu "intelligent" grâce à JavaScript, qui adapte les champs selon que l'on choisisse un film ou une série.

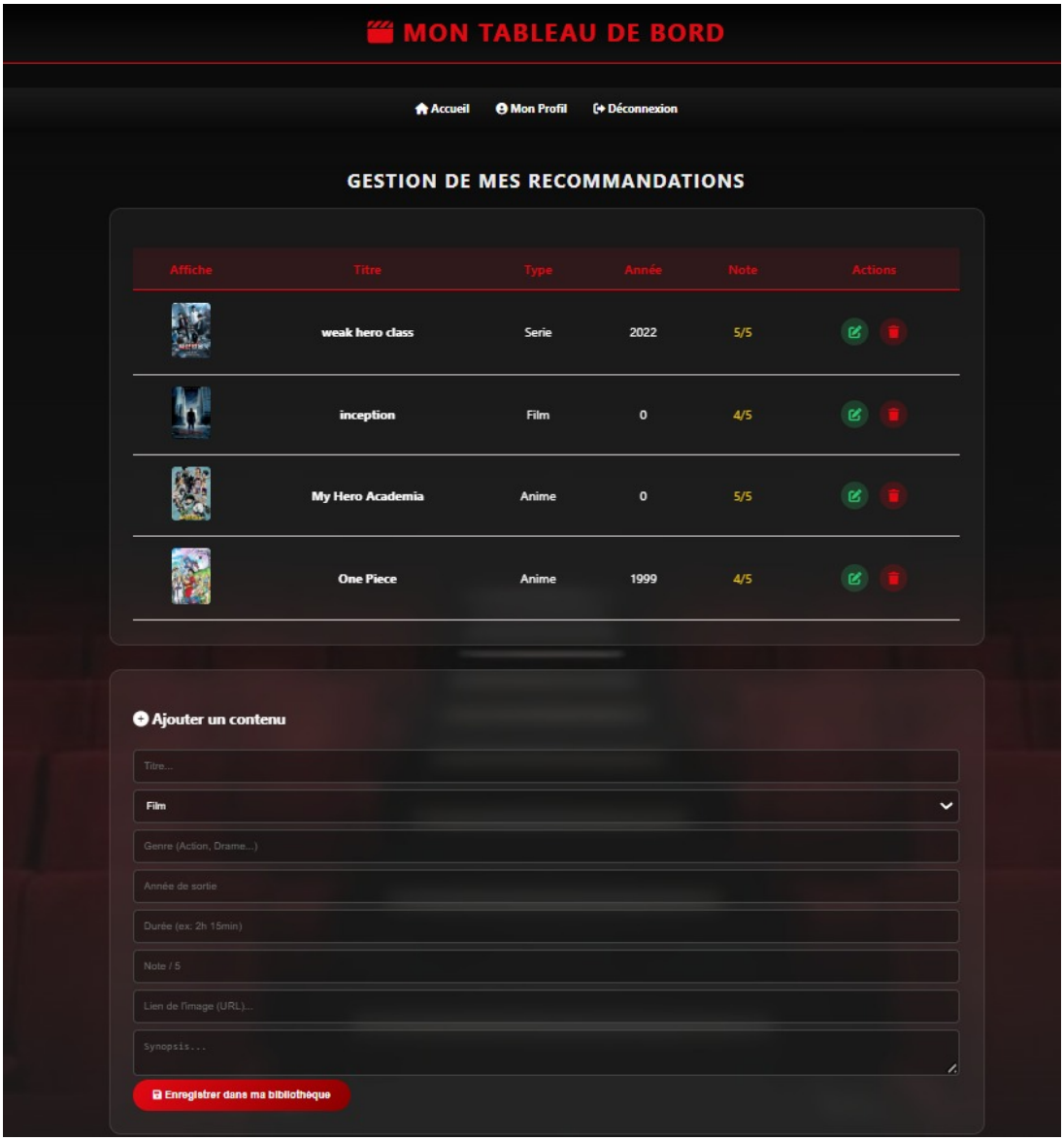

FIGURE 7 – Tableau de bord membre et formulaire d'ajout

6.3 Fiche détaillée et Avis

Lorsqu'un utilisateur clique sur une œuvre, il accède à ses détails et à l'espace de discussion (Figure 8). C'est ici que les membres peuvent publier, modifier ou supprimer leurs avis avec un système de notation par étoiles.

FIGURE 8 – Page de détail d'une œuvre et système de commentaires

6.4 Panel Administrateur

L'interface de modération (Figure 9) est strictement réservée au rôle "admin". Elle permet la gestion globale du site, incluant la suppression des contenus inappropriés et le bannissement des utilisateurs.

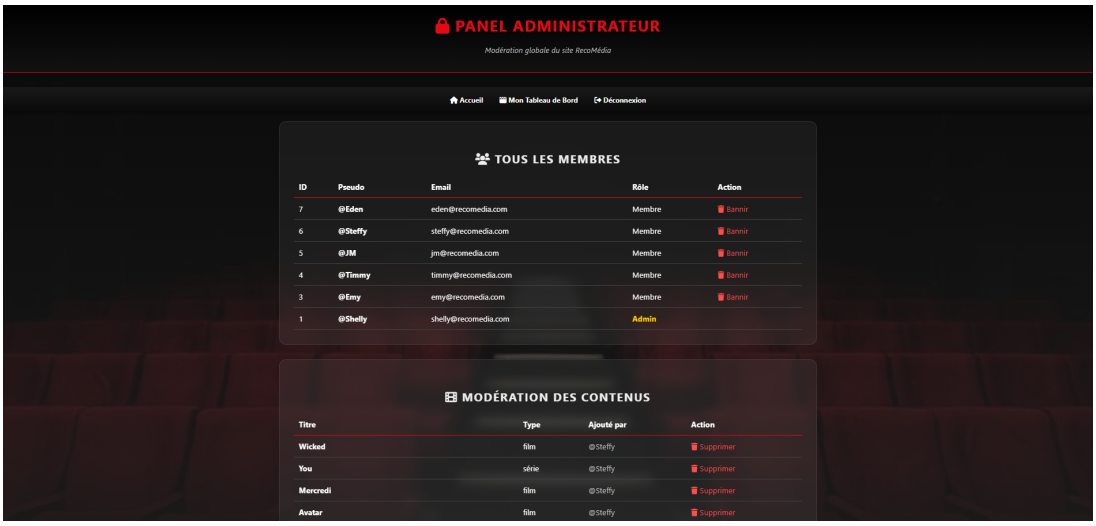

FIGURE 9 – Interface sécurisée réservée à l'administrateur

7 Difficultés rencontrées et Solutions

La réalisation de ce projet "Full Stack" en totale autonomie a soulevé plusieurs défis techniques que j'ai dû surmonter, allant de la configuration du serveur jusqu'à l'expérience utilisateur :

7.1 Défis d'infrastructure et serveur

— **Conflit de port sur le serveur de base de données :** Lors de la configuration initiale de l'environnement de développement, la connexion à la base MySQL échouait (le port par défaut 3306 étant bloqué ou occupé par un autre service). *Solution :* J'ai dû reconfigurer le serveur local pour écouter sur le port 3307, et modifier ma fonction PHP `mysqli_connect()` en lui passant explicitement ce nouveau paramètre de port.

— **Les erreurs de redirection et le cycle des sessions :** Lors de la mise en place des redirections après connexion ou déconnexion (`header("Location: ...")`), j'ai été confrontée à l'erreur fatale *"Cannot modify header information"*. *Solution :* J'ai dû restructurer mes fichiers pour garantir que la fonction `session_start()` et les traitements d'en-tête soient systématiquement placés au tout début des scripts, avant le moindre affichage HTML.

7.2 Défis de logique applicative et UX

— **La gestion de l'intégrité référentielle lors des suppressions :** Lors du bannissement d'un utilisateur depuis le panel administrateur, je risquais de créer des contenus "orphelins" dans la base de données. *Solution :* J'ai développé un script PHP séquentiel qui supprime d'abord les avis de l'utilisateur, puis ses recommandations, et enfin son compte, reproduisant ainsi une logique de suppression en cascade sécurisée.

— **L'interactivité du formulaire sans rechargement :** L'affichage des champs "Saisons" et "Épisodes" devait être conditionné par le choix "Série" ou "Film" dans le menu déroulant. *Solution :* J'ai intégré un script JavaScript (`toggleInputs()`) écoutant les changements de la balise `<select>` pour modifier dynamiquement l'attribut `display` du DOM sans avoir à recharger la page PHP.

8 Conclusion et Perspectives

8.1 Bilan du projet

Le développement de **RecoMédia** a été une expérience extrêmement formatrice, me permettant d'appliquer concrètement l'ensemble des concepts abordés en cours de programmation Web. De la conception UML rigoureuse à l'implémentation de la base de données (MySQLi), en passant par la structuration des sessions PHP et du rendu Front-end, j'ai pu couvrir et maîtriser le spectre complet du développement "Full Stack".

Le système remplit son cahier des charges : offrir une plateforme fluide, sécurisée et interactive pour partager des recommandations cinématographiques.

8.2 Perspectives d'évolution

Si l'application devait être étendue, plusieurs pistes d'évolution seraient pertinentes :

— **Upload d'images natif :** Remplacer le champ texte de l'URL par un véritable téléversement de fichier local (via l'attribut `enctype="multipart/form-data"`) géré avec la superglobale `$_FILES` en PHP.

— **Consommation d'une API externe :** Connecter le site à l'API de *TMDB (The Movie Database)* pour importer automatiquement les données officielles (affiches, synopsis) afin de faciliter la saisie.

— **Système de pagination :** Mettre en place un système de pagination (`LIMIT` et `OFFSET` en SQL) sur la page d'accueil pour optimiser les performances lorsque le catalogue de recommandations s'agrandira.

9 Bibliographie et Webographie

- **Documentation officielle PHP :** Manuel PHP et fonctions natives (ex : sessions, `password_hash`) - `https://www.php.net/docs.php`
- **Documentation MySQL :** Référence du langage SQL et de l'extension `MySQLi` - `https://dev.mysql.com/doc/`
- **Création graphique :** Le logo original du projet (pellicule et cœur) a été conçu sur mesure à l'aide de la plateforme de création graphique Canva - `https://www.canva.com/`
- **Ressources photographiques et iconographiques :** L'image d'arrière-plan du site (thème cinéma) a été sourcée via Pinterest (utilisée à des fins strictement pédagogiques). Utilisation de la bibliothèque FontAwesome pour les icônes de l'interface - `https://fontawesome.com/`
- **Ressources typographiques :** Polices d'écriture importées depuis Google Fonts.